Towards Predictive Schema Intelligence: Predicting Downstream SQL-Pipeline Failures from Schema Changes on an Execution-Grounded Synthetic Benchmark

Preprint, not peer reviewed. All experiments use synthetic data. Numbers in this document are inserted automatically from `results/` by `paper/build_manuscript.py`.

Author: Abhay Malhari Davale
Affiliation: Independent Researcher, Kolhapur, Maharashtra, India
Corresponding e-mail: davaleabhay@gmail.com

Abstract

Changes to upstream database schemas can break downstream data pipelines, and common safeguards such as schema-compatibility rules and data validation either flag changes regardless of whether anything downstream depends on them or react only after data has arrived. This paper asks whether the characteristics of a proposed schema change, together with static lineage information, can predict pipeline failure before the pipeline runs. We build a reproducible, fully synthetic, execution-grounded benchmark: 500 synthetic pipelines, each with 3–8 downstream SQL jobs, and 10000 proposed schema changes (column drop, rename, add, type change, nullability relax/tighten). Every change is applied and every job executed in DuckDB 2.0.0, so failure labels are observed outcomes rather than hand-written rules; 9614 changes were executable and 386 were rejected by the engine at migration time and excluded. Under pipeline-grouped 5-fold cross-validation repeated three times, tree-based models using change features plus static-lineage features reach F1 ≈ 0.999 ± 0.001 (Random Forest), compared with 0.797 ± 0.009 for an a-priori lineage-aware rule baseline, 0.587 ± 0.012 for a schema-only rule, and 0.680 ± 0.015 for a Random Forest restricted to change features. The advantage over the lineage-aware rule is concentrated in type changes, where the rule over-flags. Performance degrades as lineage is hidden (Random Forest F1 0.795 when each job is missing from lineage with probability 0.5), and models trained without a change type do not reliably generalize to it. The near-perfect scores show that failures in this benchmark are close to a deterministic function of

change, lineage and engine semantics; the results are specific to the synthetic generator and to DuckDB and do not establish performance on real pipelines.

Keywords: schema evolution; data pipelines; failure prediction; data quality; impact analysis; synthetic benchmark; reproducibility

1. Introduction

Data pipelines depend on the structure of their sources. When an upstream table renames or drops a column, changes a type, or alters nullability, jobs downstream may fail, or may continue and deliver unexpected results. Empirical evidence from mined GitHub projects and Stack Overflow posts attributes about a third of data-related pipeline issues to incorrect data types, and identifies compatibility issues as a distinct problem area [1]. A schema change is not, however, necessarily harmful: adding a nullable column or widening an integer to a larger integer often breaks nothing. A useful early-warning mechanism should therefore separate potentially breaking from harmless changes in the context of the pipeline that consumes them.

This paper studies a deliberately narrow version of the problem: given a proposed schema change and what is known about downstream jobs *before* anything runs, predict whether at least one downstream job will fail. Because no suitable public dataset of schema-change outcomes was found (Section 6), we construct a controlled synthetic benchmark whose labels come from actually executing SQL, and we evaluate simple, interpretable models against rule baselines with a leakage-aware protocol.

Contributions (primarily empirical and methodological, and modest in scope): (i) an open, reproducible generator and executor that produces execution-grounded failure labels for schema changes; (ii) a pre-execution feature set and a leakage-controlled evaluation protocol (pipeline-grouped splits, outcome columns excluded, tests); (iii) an empirical comparison of rule baselines and four classifiers, with feature-group ablation, leave-one-change-type-out, class-prevalence and lineage-incompleteness analyses; (iv) an honest account of why high synthetic scores should not be read as real-world evidence.

2. Background and Motivation

Schema evolution and its impact. Curino et al. analysed the schema history of the MediaWiki database behind Wikipedia using a set of schema modification operators, and proposed the case as the kernel of a schema-evolution benchmark [2]. Qiu et al. mined the histories of ten open-source database applications (over 160K revisions) with a taxonomy of 24 atomic schema-change types, and reported that schemas change frequently, that adding tables, adding columns and changing column datatypes are among the most frequent changes, and that schema changes are accompanied by substantial code modification [3]. Maule et al. proposed static program analysis (program slicing plus context-sensitive dataflow analysis) to estimate

how relational schema changes affect object-oriented applications, and evaluated it on 62 versions of a commercial content-management system, reporting no false negatives and a low level of false positives [4]. They also distinguish impacts that cause errors from those that merely change behaviour, a distinction we reflect in our scope.

Data validation and quality. Breck et al. describe a schema-based validation system for data fed to ML pipelines, deployed in production in TFX [5]. Schelter et al. present declarative “unit tests for data” executed on Spark, using ML for constraint suggestion and anomaly detection in quality-metric time series; their focus is on the quality of data values rather than on schema changes [6].

Lineage and impact analysis. LineageX extracts column-level lineage from SQL queries by static parsing and demonstrates impact analysis of upstream column changes [7]. This suggests that the static-lineage features we simulate could in principle be derived from SQL, which we did not test.

Failure and defect prediction. Jassas and Mahmoud compared seven classical classifiers on three public cluster traces (Google, LANL Mustang and Trinity) using a 75%/25% split, and reported decision trees and random forests as best, with F1 up to about 0.99 on the Google trace after feature selection [8]. In software engineering, just-in-time defect prediction uses characteristics of a *change* (for example, size and developer experience) to flag risky changes before problems are found [9], a conceptual precedent for predicting from change characteristics.

3. Related Work and Research Gap

Closest work and difference. Static impact analysis [4] and schema-based or constraint-based validation [5,6] decide from declared structure whether a change or data batch is acceptable; they do not learn from observed execution outcomes. Co-evolution studies [3,2] characterize what changes and what code co-changes, not whether a downstream execution fails. The pipeline-quality study [1] taxonomizes causes but does not predict. Job-failure and defect prediction [8,9] share the classification setup but use resource or code-change features, not schema-change characteristics. In our own reading of [8] (not a claim by its authors), identifier-like fields such as job ID, task index and machine ID were among the top-ranked features and evaluation used a single random-style split; such features can inflate scores, which is why we exclude identifiers and split by pipeline.

The closest approach we found is a recent preprint on “software world models” [10]. It trains models on labels obtained by *executing* candidate code changes in controlled systems to predict which downstream components a change will break before it is executed, and compares against a dependency-graph (typed-reachability) baseline. Its reported finding that learned prediction helps most where the extracted dependency graph is incomplete parallels our lineage-incompleteness analysis. It differs from this work in setting

(multi-repository software systems and services rather than SQL data pipelines), in model family (fine-tuned language models rather than lightweight classical models) and in change taxonomy (code and interface changes rather than column-level schema changes). It is a preprint and has not been peer reviewed to our knowledge.

Name overlap. A book chapter indexed as a “predictive approach” to database schema evolution proposes anticipating future requirement changes during schema design so that evolution can be planned and costed, according to its abstract [11]. That is design-time anticipation of change, not prediction of whether a downstream execution will fail.

Gap (stated cautiously). Among the sources we examined, we did not find a peer-reviewed study that predicts downstream SQL-pipeline failure from column-level schema-change characteristics and static lineage using execution-derived labels. The execution-labelling idea itself is not new [10]; the contribution here is its transfer to schema changes in SQL pipelines, with an open benchmark and leakage-aware protocol. The search is informal and incomplete: for data contracts and data observability we located mainly industry or grey literature, and academic sources there still need to be searched (see `paper/literature_matrix.md`). Direct queries on column-level lineage and on public schema corpora [7,12] did not reveal a study of this kind either.

4. Research Questions and Objectives

Objective. Evaluate, on an open execution-grounded synthetic benchmark, whether schema-change characteristics and static lineage can predict downstream SQL-pipeline failure before execution, and document the limits of that claim.

- **RQ1.** Which types of schema change are associated with pipeline failure in this environment? *Analysis:* failure rate by change type and type class (Table 1, Fig. 1).
- **RQ2.** Which schema-change characteristics and pipeline metadata help predict failure? *Experiments:* feature-group ablation S vs. S+D vs. S+D+P (Table 2, Fig. 3) and permutation importance (Table 7).
- **RQ3.** Can ML models identify risky changes better than simple rule baselines? *Experiment:* rules R1/R2 vs. four classifiers on identical grouped folds (Tables 2-4).
- **RQ4.** How could a predictive component integrate with existing data-engineering workflows? *Conceptual only:* a proposed design is described in Section 5; no integration was implemented or tested, so no empirical claim is made.
- **RQ5.** What are the limitations, false-positive risks and data requirements? *Analyses:* leave-one-change-type-out (Table 5), class prevalence (Table 6), incomplete lineage (Table 8), calibration (Table 9), plus Sections 11-12.

No formal hypothesis tests are performed. Scope exclusions: adding NOT NULL columns, key/constraint changes beyond nullability, nested structures, data-value drift, silent semantic changes (outputs that change without an error), real orchestrators and any production logs.

5. Proposed Framework and What Is Implemented

Problem formulation. Let a pipeline consist of a source schema SS and a set of downstream jobs JJ. A proposed change δ\delta yields a new schema S′=δ(S)S' = \delta(S). The target is y(δ,J)=1y(\delta, J) = 1 if at least one job in JJ raises an execution error when run on S′S' (otherwise 0). We learn a function y^=f(ϕ(δ,J,π(S)))\hat{y} = f(\phi(\delta, J, \pi(S))), where ϕ\phi extracts pre-execution features and π(S)\pi(S) is a profile of the current data. Failure types are handled as follows: *execution/transformation failure* (exception in a job) is the labelled target; *schema-compatibility failure* is approximated by rule baselines R1/R2; *data-quality violations* and *incorrect output without an error* are not labelled and are excluded. There is no time window: jobs run immediately after the change.

Prediction time. A schema change has been proposed but not applied and no downstream job has run. Available inputs: the change description; the declared downstream jobs and which columns each uses (static lineage); and a profile of the current data. Unavailable and never used: job results, error messages, failed-job counts.

Output. A failure probability for the (change, pipeline) pair; a threshold of 0.5 gives a binary warning.

Component	Status in this paper
Change representation, pre-execution features, models, grouped evaluation, baselines	**Implemented and evaluated** (synthetic)
Execution-grounded benchmark generator and DuckDB executor	**Implemented and evaluated**
Integration with schema registries, orchestrators (e.g., Airflow, dbt), CI hooks	Proposed only, not implemented
Training on real execution history; online monitoring; deployment	Proposed only, not implemented

No production system exists; the implementation is an experimental prototype.

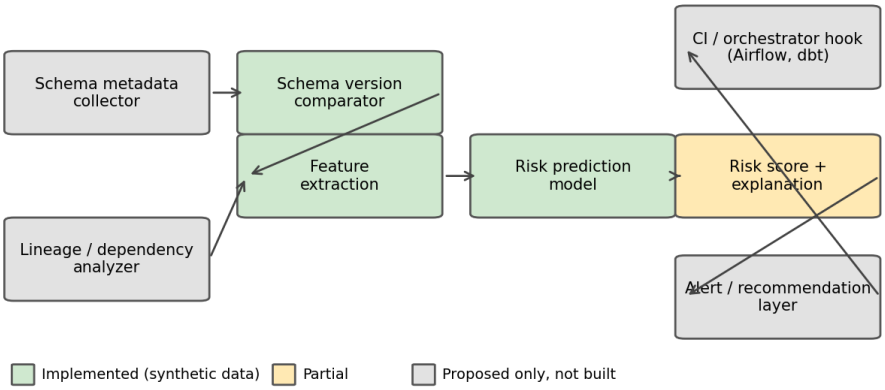


Fig. 0 (conceptual). Proposed architecture; colours show which parts exist in this study (diagram source: `paper/architecture.mmd`). The risk score is the model's predicted failure probability; no separate formula is proposed. Its calibration is assessed only on synthetic data (Table 9). *Integration (RQ4, proposed).* A realistic hook would run the predictor when a migration is proposed (for example in continuous integration), reading declared lineage from the catalog, and would warn rather than block, because false positives (Tables 3-4) have a different cost from false negatives. Whether this works with real orchestrators is untested.

6. Dataset and Experimental Setup

All data are synthetic. Each of 500 pipelines has one source table with 150 rows and 6–12 columns of types INTEGER, BIGINT, DOUBLE, VARCHAR, DATE, BOOLEAN (with nullability, null rates and numeric-like string columns varied), and 3–8 downstream jobs drawn from nine templates (projection, aggregation, filter, string function, date function, join, NOT-NULL load, positional `SELECT *` load, and a chained job that reads another job's output). Up to 20 candidate changes per pipeline are sampled uniformly from the change space; 10000 changes were proposed. A fresh schema is built per change, the change is applied via `ALTER TABLE` (type changes use `TRY_CAST`; a nullability relaxation is followed by new rows containing NULLs), and every job is executed in DuckDB [13]. **Label:** failed = 1 if at least one job raises an exception. The six change types correspond to atomic types in the taxonomy of Qiu et al. [3]: add column (A2), drop column (A6), rename column (A7), change column datatype (A8), make column NOT NULL (A21) and drop column NOT NULL (A22). Our sampling frequencies are a design choice and are not claimed to match real-world frequencies. Generator validity is tested: each pipeline must run with zero failures before any change. 386 type changes were rejected by the engine during migration and excluded; 9614 executed examples remain, with an observed failure rate of 40.9%. This base rate follows from how changes were sampled and is not an estimate of real-world prevalence. We did not find a public dataset combining schema-change history with execution outcomes; the largest public schema corpus we located, SchemaPile (221,171

schemas mined from SQL files on GitHub), describes schema structure and metadata rather than pipeline outcomes, according to its abstract [12]. It could supply realistic schemas for future work.

Table 1. Executed examples and observed failure rate by change type.

change_type	executed	failure_rate	migration_rejected
type_change	4239	0.136	386
drop_column	1875	0.811	0
rename_column	1846	0.593	0
nullability_relax	760	0.568	0
add_column	622	0.492	0
nullability_tighten	272	0.000	0

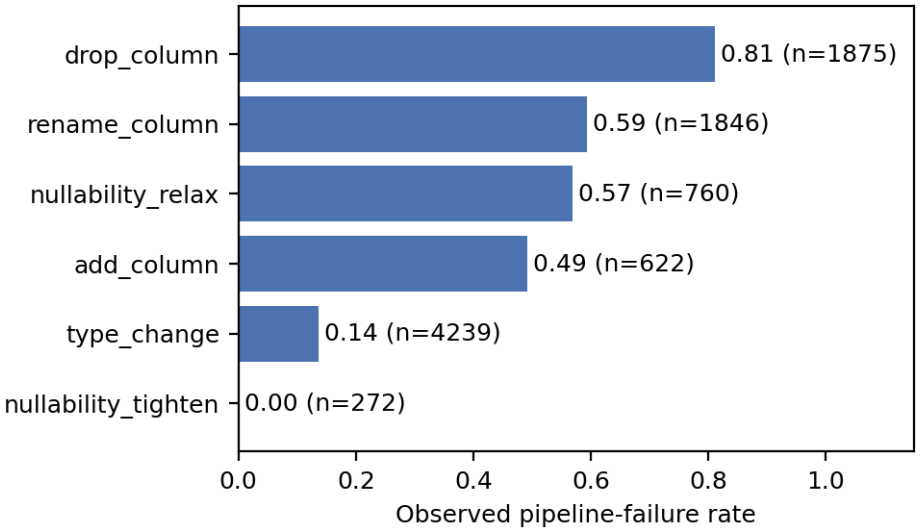


Fig. 1. Observed failure rate by change type (synthetic benchmark).

7. Feature Engineering

Three groups, usable at prediction time: **S** (schema-change; 30 features): change type, old/new type, type-compatibility class (widening, narrowing, to-string, from-string, other), nullability before/after, relative column position, table width. **D** (static lineage; 43 features cumulative): number of jobs, number of jobs using the changed column, count by usage role (select, group, measure, filter, string function, date function, join key, NOT-NULL target), number of positional jobs, transitive jobs affected through chains, maximum chain depth. **P** (pre-change profile; 45 features cumulative): null rate of the column and the fraction of its non-null values that fail to cast to the proposed type. Outcome columns are excluded by construction and by an assertion and tests.

Table F. Candidate features and availability at prediction time (all groups are computable before any job runs).

Group	Features	How extracted	Why it might help	Limitation
S	change type, old/new type, type class, nullability before/after, position, table width	From the change description and current schema	Type class and change kind determine compatibility	Does not know downstream use
D	job counts, uses by role, positional jobs, transitive jobs, depth	From declared job metadata (simulated static lineage)	Shows whether and how the column is used	Real lineage is noisier (Table 8)
P	null rate, cast-failure rate	SQL profile of current data	Data-dependent casts and NULLs	Small effect here

Not used (not available or not valid): job outcomes, error messages, failed-job counts, historical failure rate (no real history exists in this study).

8. Experimental Methodology

Baselines (fixed before experiments, never tuned on labels). **R1** (schema-only, registry-style): breaking if drop, rename, or non-widening type change. **R2** (lineage-aware): R1-style logic applied only where the change touches something downstream (explicit column use or a positional job), plus add-column breaks positional jobs and nullability relaxation breaks NOT-NULL targets or positional jobs. **Always-fail** gives a reference for the base rate.

Models. Logistic Regression (standardized), Decision Tree (depth 10), Random Forest (200 trees), HistGradientBoosting; hyper-parameters fixed a priori, no tuning.

Splits. Grouped 5-fold cross-validation with the pipeline as the group (no pipeline appears in both train and test), repeated with three seeds (42, 43, 44): 15 held-out evaluations per method. Tables report mean ± standard deviation over these 15 evaluations; because the repetitions reuse the same data these are descriptive and **no significance claims are made**. Metrics: precision, recall, F1 (failure = positive), PR-AUC, ROC-AUC, FPR, FNR, confusion matrices; because the base rate is far from rare in this benchmark, we also examine lower prevalence explicitly. Secondary analyses: leave-one-change-type-out (train on other change types, test pipelines held out by fold); prevalence sensitivity by sub-sampling failures to 20%, 10%, 5%; lineage incompleteness, where each job is hidden from the dependency features with probability q while labels stay unchanged.

9. Results

All numbers below come directly from `results/`.

Table 2. Main comparison (grouped 5-fold CV × 3 seeds; failure is the positive class).

Method	Features	Precision	Recall	F1	PR-AUC	ROC-AUC	FPR	FNR
Always-fail	-	0.409 ± 0.018	1.000 ± 0.000	0.580 ± 0.018	-	-	1.000 ± 0.000	0.000 ± 0.000
Rule R1 (schema-only)	-	0.460 ± 0.015	0.813 ± 0.018	0.587 ± 0.012	-	-	0.659 ± 0.011	0.187 ± 0.018
Rule R2 (lineage-aware)	-	0.662 ± 0.013	1.000 ± 0.000	0.797 ± 0.009	-	-	0.353 ± 0.020	0.000 ± 0.000
Logistic Regression	S	0.685 ± 0.029	0.757 ± 0.020	0.719 ± 0.017	0.763 ± 0.023	0.852 ± 0.009	0.240 ± 0.019	0.243 ± 0.020
Logistic Regression	S+D	0.938 ± 0.011	0.882 ± 0.014	0.909 ± 0.009	0.949 ± 0.008	0.972 ± 0.003	0.040 ± 0.006	0.118 ± 0.014
Logistic Regression	S+D+P	0.938 ± 0.011	0.882 ± 0.014	0.909 ± 0.009	0.949 ± 0.008	0.972 ± 0.003	0.040 ± 0.006	0.118 ± 0.014
Decision Tree	S	0.659 ± 0.026	0.759 ± 0.027	0.705 ± 0.014	0.713 ± 0.024	0.829 ± 0.010	0.271 ± 0.022	0.241 ± 0.027
Decision Tree	S+D	0.998 ± 0.002	0.999 ± 0.002	0.999 ± 0.001	0.999 ± 0.001	1.000 ± 0.001	0.001 ± 0.001	0.001 ± 0.002
Decision Tree	S+D+P	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
Random Forest	S	0.666 ± 0.029	0.697 ± 0.023	0.680 ± 0.015	0.715 ± 0.030	0.827 ± 0.010	0.242 ± 0.023	0.303 ± 0.023
Random Forest	S+D	1.000 ± 0.000	0.997 ± 0.002	0.999 ± 0.001	1.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.003 ± 0.002
Random Forest	S+D+P	1.000 ± 0.000	0.998 ± 0.002	0.999 ± 0.001	1.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.002 ± 0.002
Gradient Boosting	S	0.673 ± 0.028	0.739 ± 0.027	0.703 ± 0.012	0.740 ± 0.020	0.842 ± 0.009	0.249 ± 0.024	0.261 ± 0.027
Gradient Boosting	S+D	0.999 ± 0.001	0.998 ± 0.002	0.999 ± 0.001	1.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.001	0.002 ± 0.002
Gradient Boosting	S+D+P	1.000 ± 0.000	0.999 ± 0.002	0.999 ± 0.001	1.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.001 ± 0.002

Random Forest with S+D+P reaches F1 0.999 ± 0.001; the lineage-aware rule R2 reaches 0.797 ± 0.009; the schema-only rule R1 0.587 ± 0.012; always predicting failure 0.580 ± 0.018. Using change features only (S), Random Forest reaches 0.680 ± 0.015, and Decision Tree 0.705 ± 0.014. Adding lineage (S+D) raises Random Forest to 0.999 ± 0.001, Decision Tree to 0.999 ± 0.001, and Logistic Regression from 0.719 ± 0.017 to 0.909 ± 0.009. Adding profile features (S+D+P) changes little (Random Forest 0.999 ± 0.001; Gradient Boosting 0.703 ± 0.012 for S vs. 0.999 ± 0.001 for S+D+P).

Table 3. Pooled out-of-fold confusion counts (seed 42 folds; 9614 examples).

Method	TN	FP	FN	TP	Precision	Recall	F1
Rule R1 (schema-only)	1940	3745	738	3191	0.460	0.812	0.587
Rule R2 (lineage-aware)	3683	2002	0	3929	0.662	1.000	0.797
Random Forest (S)	4283	1402	1179	2750	0.662	0.700	0.681
	5684	1	10	3919	1.000	0.997	0.999

Method	TN	FP	FN	TP	Precision	Recall	F1
Random Forest (S+D)							
Random Forest (S+D+P)	5685	0	10	3919	1.000	0.997	0.999

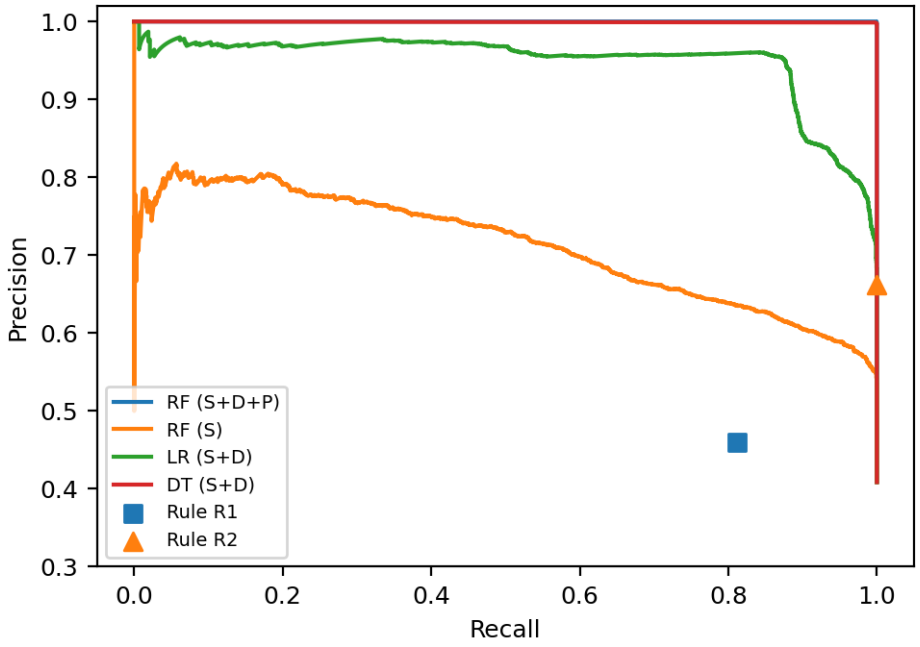


Fig. 2. Pooled out-of-fold precision–recall curves; rules shown as single points.

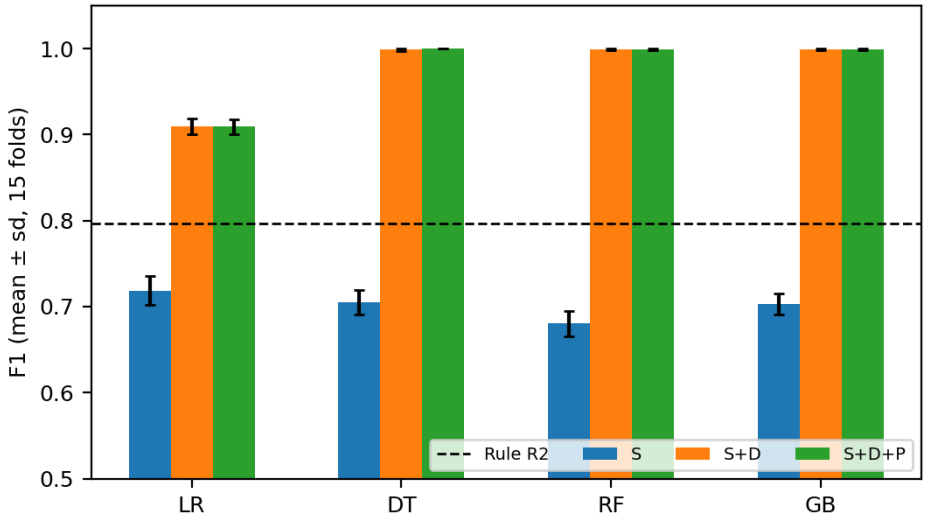


Fig. 3. F1 by model and feature set (error bars: sd over 15 evaluations); dashed line is Rule R2.

R2 produced no false negatives but 2002 false positives, all of them type changes (2002); the Random Forest (S+D+P) produced 0 false positives and 10 false negatives.

Table 4. Type-change examples only (4239 examples, 577 failures).

Method	n	failures	TN	FP	FN	TP	Precision	Recall	F1
Rule R1 (schema-only)	4239	577	1024	2638	0	577	0.179	1.000	0.304
Rule R2 (lineage-aware)	4239	577	1660	2002	0	577	0.224	1.000	0.366
Random Forest (S)	4239	577	3451	211	482	95	0.310	0.165	0.215
Random Forest (S+D)	4239	577	3661	1	10	567	0.998	0.983	0.990
Random Forest (S+D+P)	4239	577	3662	0	10	567	1.000	0.983	0.991

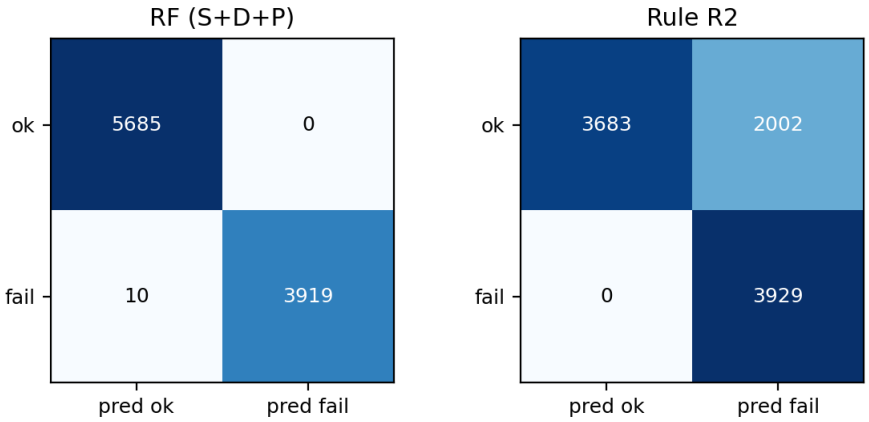


Fig. 4. Pooled confusion matrices.

Table 5. Leave-one-change-type-out F1 (training excludes the held-out change type; `nullability_tighten` has no failures, so F1 is 0 by definition).

held_out	Decision Tree	Logistic Regression	Random Forest	Rule R2 (lineage-aware)	failures	n
add_column	1.0	0.0	0.0	1.0	306	622
drop_column	0.042	0.469	0.603	1.0	1520	1875
nullability_relax	0.811	0.528	0.632	1.0	432	760
nullability_tighten	0.0	0.0	0.0	0.0	0	272
rename_column	0.425	0.69	0.547	1.0	1094	1846
type_change	0.243	0.293	0.272	0.366	577	4239

Table 6. Prevalence sensitivity (failures sub-sampled; mean over 10 draws).

prevalence	model	precision	recall	f1	pr_auc
0.05	Random Forest S	0.129	0.698	0.218	0.169
0.05	Random Forest S+D+P	1.0	0.997	0.998	1.0
0.05	Rule R1 (schema-only)	0.062	0.823	0.115	nan
0.05	Rule R2 (lineage-aware)	0.13	1.0	0.23	nan

prevalence	model	precision	recall	f1	pr_auc
0.1	Random Forest S	0.241	0.704	0.359	0.296
0.1	Random Forest S+D+P	1.0	0.997	0.998	1.0
0.1	Rule R1 (schema-only)	0.121	0.816	0.211	nan
0.1	Rule R2 (lineage-aware)	0.24	1.0	0.387	nan
0.2	Random Forest S	0.414	0.698	0.52	0.479
0.2	Random Forest S+D+P	1.0	0.998	0.999	1.0
0.2	Rule R1 (schema-only)	0.235	0.811	0.365	nan
0.2	Rule R2 (lineage-aware)	0.415	1.0	0.587	nan
natural	Random Forest S	0.662	0.7	0.681	0.712
natural	Random Forest S+D+P	1.0	0.997	0.999	1.0
natural	Rule R1 (schema-only)	0.46	0.812	0.587	nan
natural	Rule R2 (lineage-aware)	0.662	1.0	0.797	nan

Table 7. Permutation importance of the top Random Forest (S+D+P) features (drop in PR-AUC, mean over held-out folds).

feature	perm_importance_pr_auc
d_n_positional	0.0645
s_change_drop_column	0.0044
d_role_string_fn	0.0043
d_n_affected	0.0030
d_role_filter	0.0028
d_role_measure	0.0017
d_role_date_fn	0.0017
d_n_direct	0.0015
s_change_nullability_tighten	0.0010
d_role_notnull_target	0.0005

Table 8. Robustness to incomplete lineage (q = probability each job is hidden from lineage features; S+D features).

Hidden-job prob. q	Method	Precision	Recall	F1	PR-AUC
0.0	Decision Tree	0.998 ± 0.002	0.999 ± 0.002	0.999 ± 0.001	0.999 ± 0.001
0.0	Logistic Regression	0.938 ± 0.011	0.882 ± 0.014	0.909 ± 0.009	0.949 ± 0.008
0.0	Random Forest	1.000 ± 0.000	0.997 ± 0.002	0.999 ± 0.001	1.000 ± 0.000
0.0	Rule R1 (schema-only)	0.460 ± 0.015	0.813 ± 0.018	0.587 ± 0.012	-
0.0	Rule R2 (lineage-aware)	0.662 ± 0.013	1.000 ± 0.000	0.797 ± 0.009	-
0.25	Decision Tree	0.943 ± 0.013	0.850 ± 0.016	0.894 ± 0.009	0.955 ± 0.006
0.25	Logistic Regression	0.820 ± 0.020	0.852 ± 0.013	0.836 ± 0.011	0.919 ± 0.011
0.25	Random Forest	0.963 ± 0.011	0.845 ± 0.020	0.900 ± 0.009	0.964 ± 0.004
0.25	Rule R1 (schema-only)	0.460 ± 0.015	0.813 ± 0.018	0.587 ± 0.012	-
0.25					-

Hidden-job prob. q	Method	Precision	Recall	F1	PR-AUC
	Rule R2 (lineage-aware)	0.666 ± 0.011	0.843 ± 0.022	0.744 ± 0.011	
0.5	Decision Tree	0.850 ± 0.018	0.740 ± 0.030	0.791 ± 0.017	0.902 ± 0.012
0.5	Logistic Regression	0.798 ± 0.020	0.789 ± 0.016	0.793 ± 0.014	0.880 ± 0.015
0.5	Random Forest	0.835 ± 0.024	0.759 ± 0.023	0.795 ± 0.015	0.916 ± 0.009
0.5	Rule R1 (schema-only)	0.460 ± 0.015	0.813 ± 0.018	0.587 ± 0.012	-
0.5	Rule R2 (lineage-aware)	0.671 ± 0.015	0.616 ± 0.030	0.642 ± 0.018	-
0.75	Decision Tree	0.746 ± 0.032	0.703 ± 0.030	0.723 ± 0.010	0.836 ± 0.011
0.75	Logistic Regression	0.732 ± 0.028	0.752 ± 0.022	0.741 ± 0.012	0.835 ± 0.013
0.75	Random Forest	0.736 ± 0.024	0.718 ± 0.023	0.726 ± 0.005	0.844 ± 0.010
0.75	Rule R1 (schema-only)	0.460 ± 0.015	0.813 ± 0.018	0.587 ± 0.012	-
0.75	Rule R2 (lineage-aware)	0.674 ± 0.023	0.356 ± 0.039	0.464 ± 0.033	-

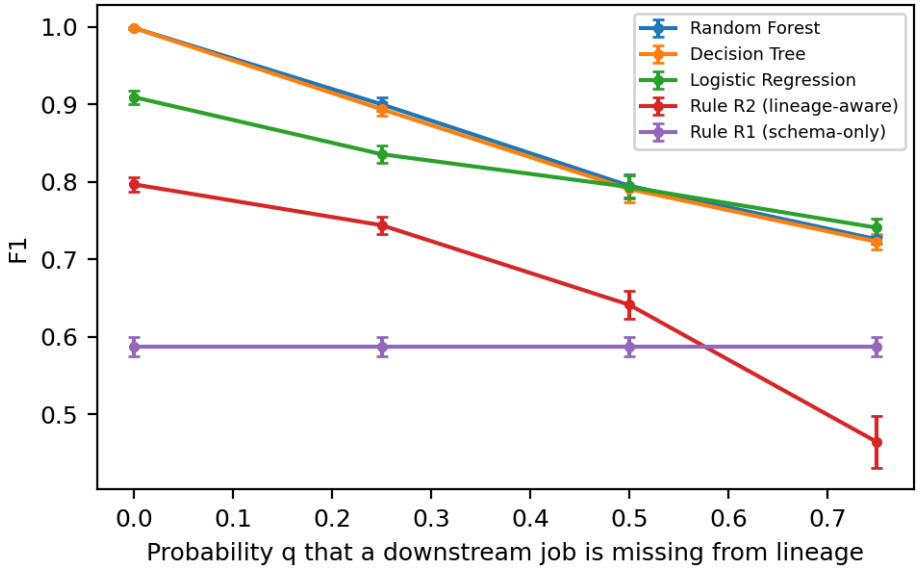


Fig. 5. F1 versus the fraction of hidden lineage.

At q = 0.25, 0.5, 0.75 the Random Forest F1 is 0.900, 0.795, 0.726, versus 0.744, 0.642, 0.464 for R2 recomputed on the same degraded features.

Table 9. Probability calibration (Brier score; lower is better; pooled out-of-fold, seed 42 folds). Calibration is examined only on synthetic data.

Predictor	Brier score
Constant prevalence (reference)	0.2417
Logistic Regression (S+D)	0.0590
Decision Tree (S+D)	0.0010
Random Forest (S)	0.1713
Random Forest (S+D+P)	0.0053
Gradient Boosting (S+D+P)	0.0003

10. Discussion

What the observations support. (a) Change characteristics alone are only moderately informative here (F1 for S-only models: Random Forest 0.680 ± 0.015, Logistic Regression 0.719 ± 0.017), and static lineage is what makes prediction accurate: knowing which columns downstream jobs use, and how, separates harmful from harmless changes. (b) The a-priori lineage-aware rule R2 already catches every failure but over-flags type changes; the trained models mainly add precision on type changes. A plausible explanation, not tested here, is that the engine's tolerance for a type change depends on the pair of types and on how the column is used (e.g., aggregation versus string function), which R2 does not encode. A rule written with an engine-specific cast table might close this gap; we did not evaluate such a rule. (c) Scores near 1.0 mean the benchmark is close to deterministic given (change, lineage, engine): the models are largely recovering the engine's semantics from examples. This is why the results should not be read as predictive capability in real settings. (d) Performance falls sharply when lineage is incomplete, and leave-one-change-type-out shows models do not reliably extrapolate to change types they have not seen, whereas rule R2, which encodes structure explicitly and is not trained, does not depend on exposure to a change type. A similar pattern, in which learned predictors help most where the extracted dependency graph is incomplete, has been reported for software systems [10]; that study is a non-peer-reviewed preprint in a different setting, so we treat it as consistent with, not confirmation of, our finding. (e) Prevalence sub-sampling shows precision of the change-only model and of the rules drops as failures become rarer, as expected for a fixed false-positive rate.

Practical implications (proposed, untested in production). If lineage metadata is available and reasonably complete, a pre-merge check that scores a schema migration could help prioritise review; lineage completeness is the main practical dependency (Table 8). A warning-only mode limits the cost of false positives. These statements are hypotheses, not findings.

Untested hypotheses and future work. Whether the same feature groups matter on real pipelines; whether results hold on other engines (Spark SQL, Postgres, BigQuery); behavior when failures are silent; learning from real execution history; comparison against an engine-aware rule engine; cost-sensitive thresholds.

11. Threats to Validity

Construct: failure is defined as an exception; silent data corruption is not measured. *Internal:* hyper-parameters were fixed, not tuned; baseline R2 was written before experiments but by the author with knowledge of the generator, which may favor it on structural change types. *External:* synthetic pipelines, one SQL engine (DuckDB 2.0.0), small tables, job templates chosen by the author, and a base rate set by the sampling design. *Statistical:* repeated CV on one dataset; spread estimates are descriptive and not independent. *Circularity:* labels are execution-based, but job templates and feature definitions share a vocabulary (roles), so lineage features describe exactly how jobs use columns; real lineage metadata is noisier.

12. Limitations

The study is a proof of concept on a synthetic benchmark. It does not include real production data, concurrent or streaming settings, multi-table schemas, or data-value drift. 386 type changes were excluded because the engine rejected them at migration; this removes cases where the database itself prevents the change. The literature review is incomplete.

13. Ethical and Reproducibility Considerations

No personal or proprietary data are used. All code, seeds, configuration and a dataset fingerprint are provided; `python src/run_all.py` regenerates all tables, figures and this manuscript; 8 automated tests cover generator determinism, generator validity, leakage guards, grouped splitting and rule semantics. The author independently re-ran the full pipeline on Windows (Python 3.14, DuckDB 2.0.0, pandas 3.1.1, NumPy 2.5.4, scikit-learn 1.9.1); the dataset fingerprint, example counts, and the precision, recall, F1 and F1 standard deviation of all 15 method/feature-set rows in Table 2, as well as the lineage-robustness F1 values quoted in Section 9, matched the values reported here to three decimals. Software: Python 3.12, DuckDB [13] 2.0.0, scikit-learn [14], pandas, NumPy, Matplotlib (versions in `results/run_config.json` and `requirements.txt`). An MIT licence file is included; archiving the repository (for example on Zenodo) would provide a citable DOI.

Data availability. All data are synthetic and regenerated by `src/build_dataset.py` (seed 42; fingerprint in `data/dataset_meta.json`). No third-party data were used. **Code availability.** Code, tests and manuscript build scripts are in the project repository (https://github.com/8abhay/predictive-schema-intelligence; MIT licence). **Funding.** None declared by the author. **Conflict of interest.** None declared by the author. **Author contributions.** Abhay Malhari Davale is the sole author. **AI-assistance disclosure.** An AI assistant (Claude, Anthropic) helped with literature search, code, experiment execution and drafting; the author must review every claim, verify every reference and take responsibility for the final text; follow the target venue's AI-use policy. **Acknowledgements.** None.

14. Conclusion and Future Work

On an execution-grounded synthetic benchmark, combining schema-change features with static lineage yields near-perfect failure prediction, clearly better than a schema-only rule and than change features alone, and more precise than an a-priori lineage-aware rule; but the result reflects the near-determinism of the benchmark, degrades with incomplete lineage and does not extend to unseen change types. The principal value of the work is a reproducible, leakage-aware evaluation scaffold and an honest baseline for future studies on real pipeline histories.

References

Reference metadata were checked against publisher, arXiv or index pages; the status of each reference is recorded in `paper/citation_audit.md`.

[1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, “Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers,” *J. Syst. Softw.*, vol. 207, art. no. 111855, 2024, doi:10.1016/j.jss.2023.111855 (preprint: arXiv:2309.07067).

[2] C. A. Curino, H. J. Moon, L. Tanca, and C. Zaniolo, “Schema evolution in Wikipedia: Toward a web information system benchmark,” in *Proc. 10th Int. Conf. Enterprise Information Systems (ICEIS)*, vol. 1, 2008, pp. 323–332, doi:10.5220/0001713003230332.

[3] D. Qiu, B. Li, and Z. Su, “An empirical analysis of the co-evolution of schema and code in database applications,” in *Proc. 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE ’13)*, Saint Petersburg, Russia, 2013, pp. 125-135, doi: 10.1145/2491411.2491431.

[4] A. Maule, W. Emmerich, and D. S. Rosenblum, “Impact analysis of database schema changes,” in *Proc. Int. Conf. Software Engineering (ICSE)*, 2008, pp. 451–460, doi:10.1145/1368088.1368150.

[5] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proc. SysML*, 2019.

[6] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, “Automating large-scale data quality verification,” *Proc. VLDB Endow.*, vol. 11, no. 12, pp. 1781–1794, 2018, doi: 10.14778/3229863.3229867.

[7] S. H. Zhang, Z. Miao, and J. Wang, “LineageX: A column lineage extraction system for SQL,” arXiv:2505.23133, 2025 (demo track, 41st IEEE Int. Conf. Data Engineering, ICDE 2025, per the arXiv comments).

[8] M. S. Jassas and Q. H. Mahmoud, “Analysis of job failure and prediction model for cloud computing using machine learning,” *Sensors*, vol. 22, no. 5, art. 2035, 2022, doi:10.3390/s22052035.

[9] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, “A large-scale empirical study of just-in-time quality assurance,” *IEEE Trans. Softw. Eng.*, vol. 39, no. 6, pp. 757–773, Jun. 2013, doi:10.1109/TSE.2012.70.

[10] T. Su, Y. Hu, L. Zhao, B. Zhu, J. Somasundaram, and H. Haque, “Software world models: From consequence prediction to decision value,” arXiv:2610.04940, 2026 (preprint, not peer reviewed to our knowledge).

[11] H. Bounif, “Predicitive [sic] approach for database schema evolution,” in *Intelligent Databases*, IGI Global, 2007, pp. 286-309, doi:10.4018/978-1-59904-120-9.ch011 (title as indexed by Crossref; abstract-level reading only).

[12] T. Döhmen, R. Geacu, M. Hulsebos, and S. Schelter, “SchemaPile: A large collection of relational database schemas,” *Proc. ACM Manag. Data*, vol. 2, no. 3 (SIGMOD), art. 172, Jun. 2024, doi: 10.1145/3654975.

[13] M. Raasveldt and H. Mühleisen, “DuckDB: An embeddable analytical database,” in *Proc. 2019 Int. Conf. Management of Data (SIGMOD ’19)*, 2019, pp. 1981-1984, doi:10.1145/3299869.3320212.

[14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011 (metadata taken from the reference list of [6]).